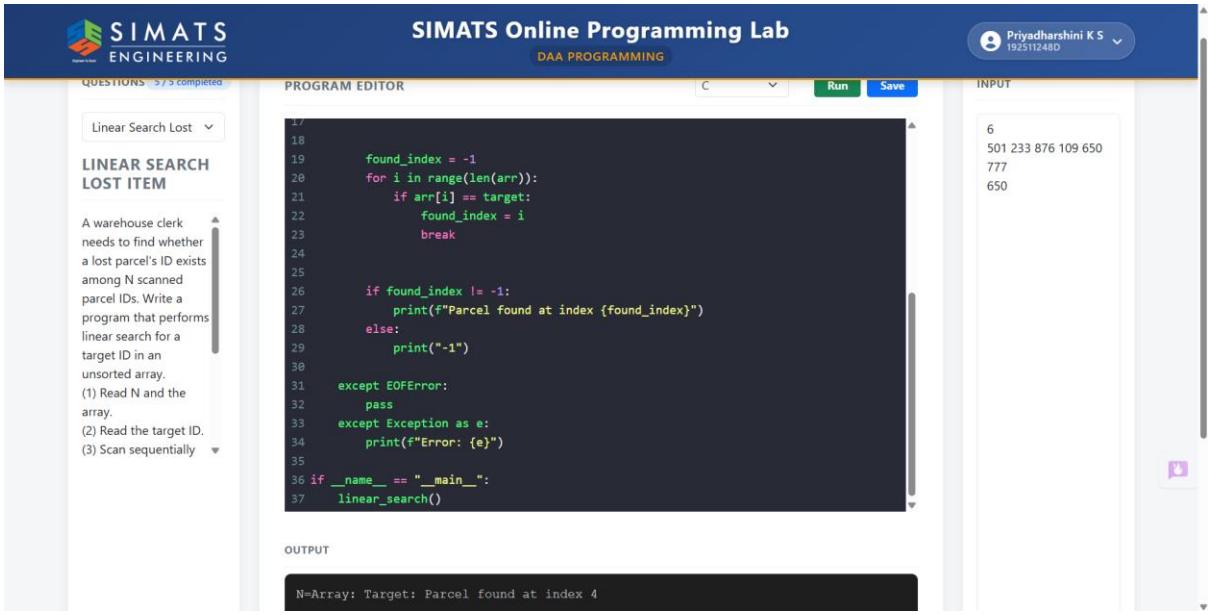
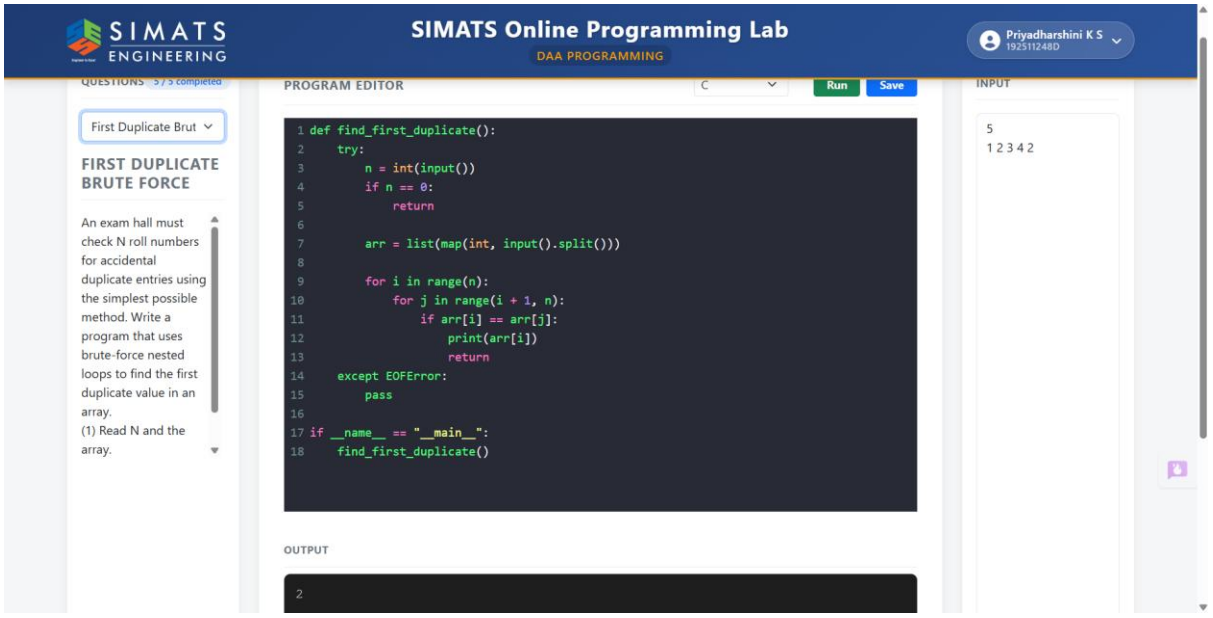
BRUTE FORCE SEARCH





SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Priyadharshini K S
192511248D

QUESTIONS 5 / 5 completed

Naive Pattern Matc

NAIVE PATTERN MATCH

A plagiarism checker needs to find every place a suspicious phrase appears inside a larger document string, using the most direct method. Write a program implementing brute-force (naive) string matching.
(1) Read the text string and the pattern string.

PROGRAM EDITOR

C

Run Save

```
    n = len(text)
    m = len(pattern)
    found_indices = []

    for i in range(n - m + 1):
        match = True
        for j in range(m):
            if text[i + j] != pattern[j]:
                match = False
                break

        if match:
            found_indices.append(i)

    if found_indices:
        print("Pattern found at indices:", " ".join(map(str, found_indices)))
    else:
        print("Pattern not found.")

if __name__ == "__main__":
    naive_pattern_match()
```

INPUT

```
ABABCABABA
ABA
```

OUTPUT

```
Text: Pattern: Pattern found at indices: 0 5 7
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Priyadharshini K S
192511248D

Max Min Brute For

MAX MIN BRUTE FORCE

A weather station logs N temperature readings and needs both the hottest and coldest reading of the day using a single straightforward scan. Write a program that finds the maximum and minimum values in an array using brute-force comparison (no

```
n = int(input())
readings = list(map(int, input().split()))

if n > 0:
    max_val = readings[0]
    min_val = readings[0]

    for i in range(1, n):
        if readings[i] > max_val:
            max_val = readings[i]
        if readings[i] < min_val:
            min_val = readings[i]

    print(f"Maximum: {max_val}")
    print(f"Minimum: {min_val}")
```

```
6
28 31 19 35 22 30
```

OUTPUT

```
Maximum: 35
Minimum: 19
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Priyadharshini K S
192511248D

Substring Existence

SUBSTRING EXISTENCE CHECK

A password strength checker wants to verify, in the simplest way, whether a known weak fragment appears anywhere inside a user's password. Write a program that checks whether a pattern string occurs as a substring of a text string using brute-

```
text = input("Text: ")
pattern = input("Pattern: ")

n = len(text)
m = len(pattern)
found = False

for i in range(n - m + 1):
    match = True
    for j in range(m):
        if text[i + j] != pattern[j]:
            match = False
            break

    if match:
        print(f"Found at index {i}")
        found = True
        break

if not found:
    print("Not found")
```

OUTPUT

```
Text: Pattern: Found at index 6
```

mypassword123
word